МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

Кафедра радіоелектронних і комп`ютерних систем

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 3

з дисципліни “Аналіз та рефакторинг коду, моделювання та проектування ПЗ”

на тему “Основи об'єктно-орієнтованого моделювання”

Виконав:

Студент 3 курсу

групи ФеП-31

Капустін Денис

Перевірив:

асистент Гура Володимир Тарасович

Львів-2026

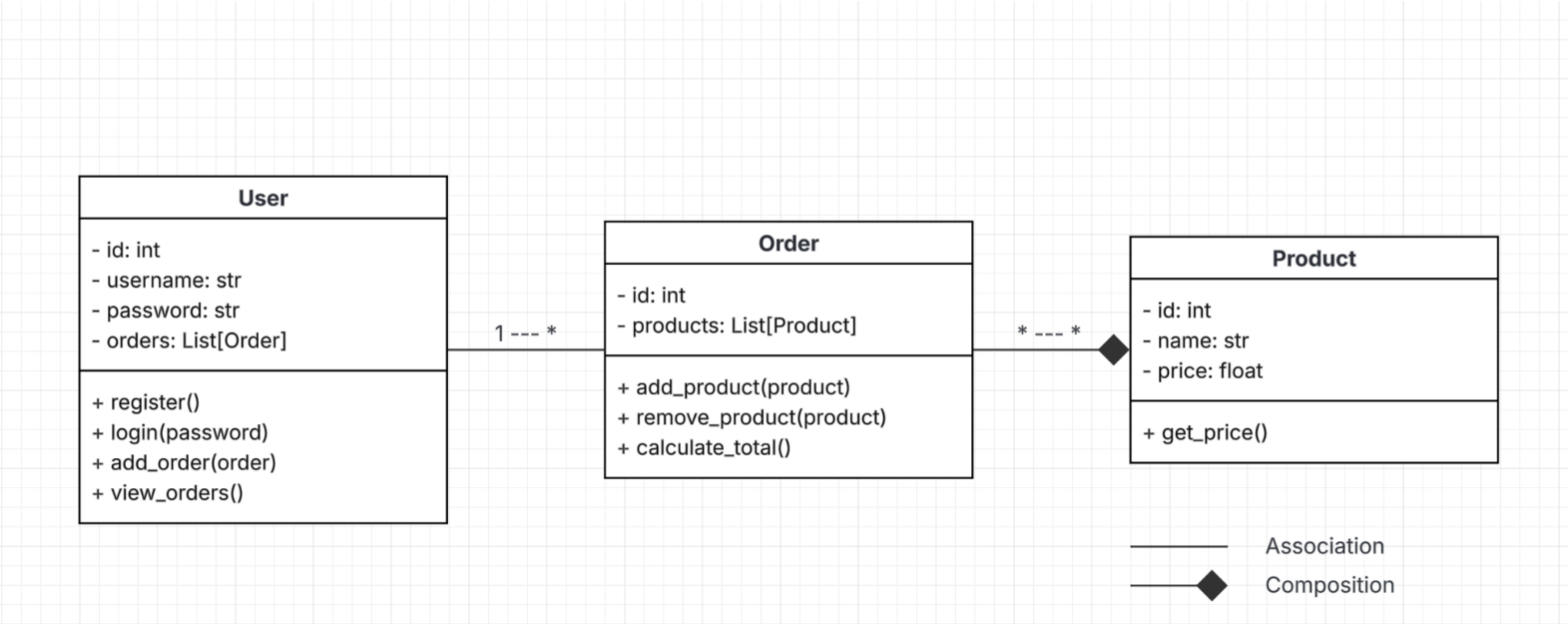
Мета роботи: Набути практичних навичок проектування архітектури програмних систем за допомогою мови моделювання UML, реалізації об’єктно-орієнтованої моделі мовою програмування високого рівня та освоїти методику модульного тестування Unit Testing для перевірки коректності роботи створеного програмного забезпечення.

Крок 1: Створення UML-діаграм

1. Діаграма класів (Class Diagram)

У ході аналізу системи «Інтернет-магазин» було виділено три ключові сутності: **User**, **Product** та **Order**.

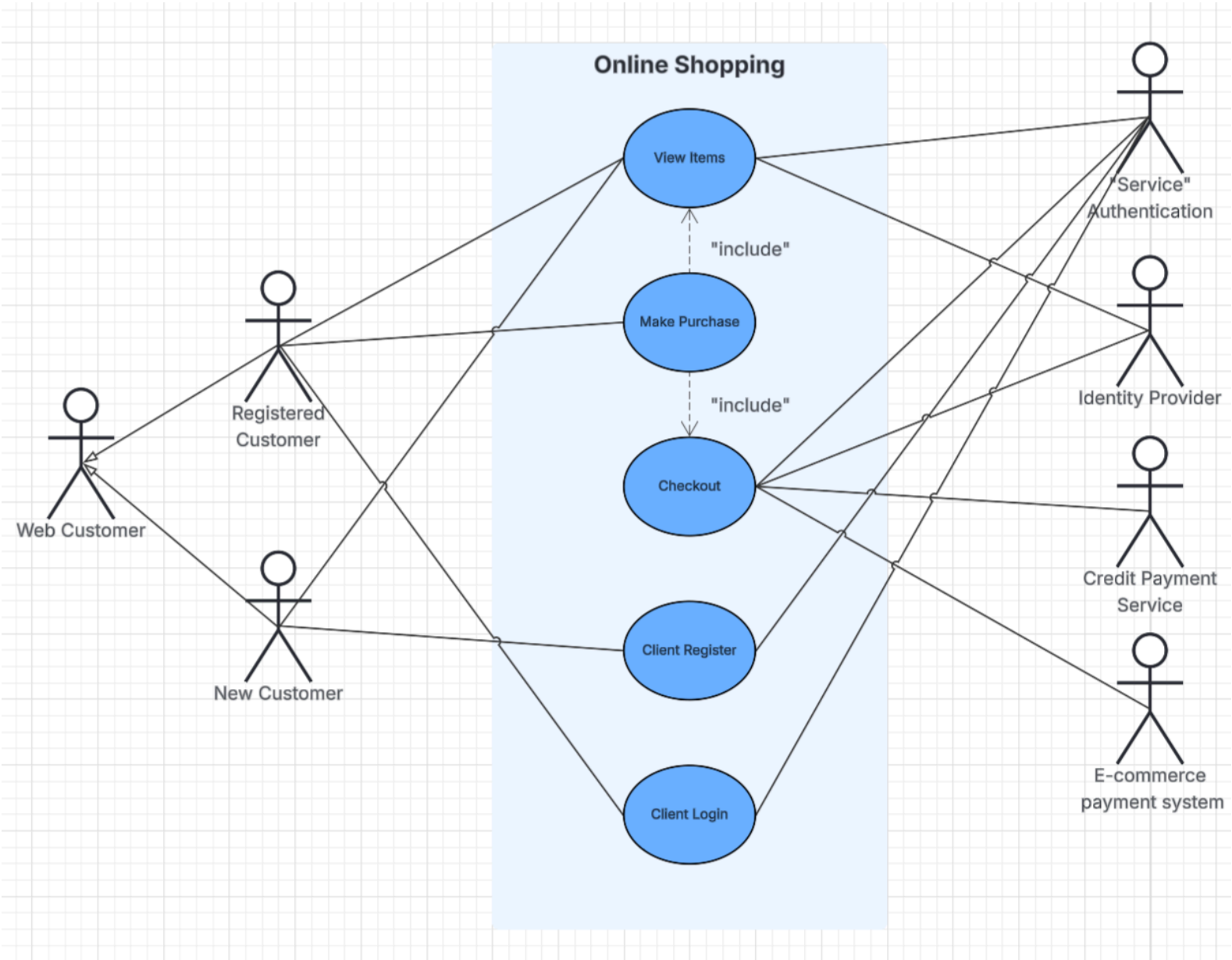
- **User (Користувач):** Містить дані про профіль та список замовлень. Пов’язаний із класом Order відношенням асоціації (один користувач може мати багато замовлень).
- **Product (Товар):** Описує характеристики одиниці товару (назва, ціна).
- **Order (Замовлення):** Виступає контейнером для товарів. Використовує агрегацію для зберігання списку об’єктів Product.



1.2. Діаграма прецедентів (Use Case Diagram)

Визначено основні сценарії взаємодії користувача (Актора) із системою:

1. **Перегляд замовлень:** відстеження товарів.
2. **Керування замовленням:** додавання та видалення товарів.
3. **Оплата:** розрахунок вартості замовлення
4. **Реєстрація користувача:** створення нового облікового запису.
5. **Авторизація:** вхід у систему за паролем.



Крок 2: Впровадження системних класів

Реалізація виконана мовою Python у файлі main.py. Кожен клас відповідає визначеним атрибутам та методам.

Клас Product

Інкапсулює дані про товар.

```
class Product:

    def __init__(self, product_id, name, price):

        self._id = product_id

        self._name = name

        self._price = price


    def get_price(self):

        return self._price
```

Клас Order

Відповідає за логіку формування замовлення та розрахунок вартості.

```
class Order:

    def __init__(self, order_id):

        self._id = order_id

        self._products = []


    def add_product(self, product):

        self._products.append(product)


    def remove_product(self, product):

        if product in self._products:

            self._products.remove(product)


    def calculate_total(self):

        return sum(p.get_price() for p in self._products)
```

```
    @property
    def products(self):
        return self._products
```

Клас User

Керує даними користувача та його замовленнями.

```
class User:
    def __init__(self, user_id, username, password):
        self._id = user_id
        self._username = username
        self._password = password
        self._orders = []

    def register(self):
        return f"User {self._username} registered"

    def login(self, password):
        return self._password == password

    def add_order(self, order):
        self._orders.append(order)

    def view_orders(self):
        return self._orders

    @property
```

```
    def orders(self):

        return self._orders
```

Крок 3: Модульне тестування

Для перевірки працездатності системи було розроблено 10 тестів у файлі tests.py, що використовують бібліотеку unittest.

№	Назва тесту	Що перевіряється
1	test_user_registration	Коректність рядка підтвердження реєстрації.
2	test_login_success	Успішний вхід з правильним паролем.
3	test_login_fail	Відмова у доступі при неправильному паролі.
4	test_add_product	Додавання об'єкта товару до списку в замовленні.
5	test_remove_product	Видалення товару та оновлення списку.
6	test_calculate_total_single	Розрахунок суми для одного товару.
7	test_calculate_total_multiple	Розрахунок загальної суми для кількох товарів.

8	test_user_add_order	Зв'язок замовлення з конкретним користувачем.
9	test_user_view_orders	Можливість перегляду списку всіх замовлень.
10	test_user_places_order	**Інтеграційний тест:** повний цикл від додавання товару до підрахунку фінальної вартості в профілі користувача.

Результат тестування:

```
[Running] python -u "e:\Projects\Python\Refactoring\Lab3\tests.py"
..........
----------------------------------------------------------------------
Ran 10 tests in 0.000s

OK

[Done] exited with code=0 in 0.156 seconds
```

Усі тести пройдено успішно, що підтверджує відповідність реалізації архітектурним вимогам.

Висновки:

Під час виконання лабораторної роботи було реалізовано систему «Інтернет-магазин». Проведено шлях від UML-моделювання до програмної реалізації та верифікації коду через Unit-тести. Система демонструє принципи низької зв'язаності та правильного розподілу відповідальностей між класами.

Додатки:

main.py

```python
class Product:
    def __init__(self, product_id, name, price):
        self._id = product_id
        self._name = name
        self._price = price

    def get_price(self):
        return self._price


class Order:
    def __init__(self, order_id):
        self._id = order_id
        self._products = []

    def add_product(self, product):
        self._products.append(product)

    def remove_product(self, product):
        if product in self._products:
            self._products.remove(product)

    def calculate_total(self):
        return sum(p.get_price() for p in self._products)

    @property
    def products(self):
        return self._products


class User:
    def __init__(self, user_id, username, password):
        self._id = user_id
        self._username = username
        self._password = password
        self._orders = []

    def register(self):
        return f"User {self._username} registered"

    def login(self, password):
        return self._password == password
```

```
    def add_order(self, order):
        self._orders.append(order)

    def view_orders(self):
        return self._orders

    @property
    def orders(self):
        return self._orders
```

tests.py

```
import unittest
import main

class TestOnlineStore(unittest.TestCase):

    def setUp(self):
        self.user = main.User(1, "denis", "1234")
        self.product1 = main.Product(1, "Laptop", 1000)
        self.product2 = main.Product(2, "Mouse", 50)
        self.order = main.Order(1)

    # 1
    def test_user_registration(self):
        self.assertEqual(self.user.register(), "User denis registered")

    # 2
    def test_login_success(self):
        self.assertTrue(self.user.login("1234"))

    # 3
    def test_login_fail(self):
        self.assertFalse(self.user.login("wrong"))

    # 4
    def test_add_product(self):
        self.order.add_product(self.product1)
        self.assertIn(self.product1, self.order.products)

    # 5
    def test_remove_product(self):
        self.order.add_product(self.product1)
        self.order.remove_product(self.product1)
```

```python
        self.assertNotIn(self.product1, self.order.products)

    # 6
    def test_calculate_total_single(self):
        self.order.add_product(self.product1)
        self.assertEqual(self.order.calculate_total(), 1000)

    # 7
    def test_calculate_total_multiple(self):
        self.order.add_product(self.product1)
        self.order.add_product(self.product2)
        self.assertEqual(self.order.calculate_total(), 1050)

    # 8
    def test_user_add_order(self):
        self.user.add_order(self.order)
        self.assertIn(self.order, self.user.orders)

    # 9
    def test_user_view_orders(self):
        self.user.add_order(self.order)
        self.assertEqual(self.user.view_orders(), [self.order])

    # 10 (взаємодія)
    def test_user_places_order(self):
        self.order.add_product(self.product1)
        self.user.add_order(self.order)

        self.assertEqual(len(self.user.orders), 1)
        self.assertEqual(self.user.orders[0].calculate_total(), 1000)


if __name__ == "__main__":
    unittest.main()
```